

Design Patterns Interview Questions - SDE Interview Prep

TOP 10 MOST IMPORTANT QUESTIONS

Q1: What are design patterns and why are they used in software development?

Answer: Design patterns are reusable and proven solutions to common software design problems. They provide a structured approach for building flexible, maintainable, and scalable applications while helping developers write cleaner code and make better architectural decisions.

Polished Answer: Design patterns are template-based, battle-tested solutions that solve recurring problems in software architecture. They help developers:

- Promote loose coupling and modular design
- Enhance code reusability, maintainability, and scalability
- Provide a common vocabulary for team communication
- Avoid reinventing the wheel for well-known problems

TL;DR: Proven, reusable blueprints for common software design problems that improve code quality and team communication.

Keyword/Key mappings: Reusable solutions | Best practices | Creational/Structural/Behavioral | Gang of Four

Q2: What are the types of Design Patterns? Explain each category with examples.

Answer: Three main types of Design Patterns:

- **Creational Patterns:** Deal with object creation mechanisms (e.g., Singleton, Factory, Builder, Prototype, Abstract Factory)
 - **Structural Patterns:** Deal with object composition and inheritance (e.g., Adapter, Bridge, Composite, Decorator, Facade, Flyweight, Proxy)
 - **Behavioral Patterns:** Deal with object interactions and communication (e.g., Observer,
-

Strategy, Command, Iterator, Visitor)

Polished Answer: Design patterns are categorized into three families based on their purpose:

1. **Creational Patterns** abstract the instantiation process, making systems independent of how objects are created, composed, and represented
2. **Structural Patterns** focus on composing classes and objects into larger structures while keeping them flexible and efficient
3. **Behavioral Patterns** are concerned with algorithms and assignment of responsibilities between objects, defining communication patterns

TL;DR: Creational = object creation, Structural = object composition, Behavioral = object interaction.

Keyword/Key mappings: Object creation | Composition | Communication | Singleton/Factory | Adapter/Facade | Observer/Strategy

Q3: Explain the Singleton Pattern. When would you use it? What are its disadvantages?

Answer: The Singleton Pattern ensures that a class has only one instance and provides a global point of access to that instance. It is used when we want to limit creation of an object to only one instance, ensuring controlled access to a resource.

Polished Answer: Singleton restricts a class to exactly one instance with a global access point. Implementation requires:

- Private constructor
- Static instance variable
- Public static getInstance() method

Use cases: Database connections, configuration managers, logging frameworks, Runtime class in Java.

Disadvantages:

- Creates global state (harder to test)
- Introduces tight coupling
- Can cause issues in multi-threaded environments if not implemented carefully
- Makes unit testing difficult (hard to mock)

Real-world example: `Runtime.getRuntime()` in Java ensures a single runtime

environment instance.

TL;DR: One instance, global access. Use for shared resources; avoid overuse due to testing/coupling issues.

Keyword/Key mappings: Single instance | Global access | Private constructor | Thread safety | Lazy initialization | Runtime class

Q4: Explain the Factory Method Pattern and how it differs from Abstract Factory Pattern.

Answer: The Factory Method Pattern defines an interface for creating objects, allowing subclasses to alter the type of objects created. This transfers object creation responsibility to subclasses, enabling flexibility and extensibility.

Polished Answer: Factory Method uses inheritance to delegate object instantiation to subclasses. The creator class defines an abstract `factoryMethod()` that concrete creators override to produce specific products.

Factory vs Abstract Factory:

- **Factory Method:** Creates one product through a single method, using inheritance
- **Abstract Factory:** Creates families of related products, using composition

Use Factory Method when:

- Object type should be determined by subclasses
- You want to centralize object creation logic
- Products share a common interface

Use Abstract Factory when:

- Multiple related products must be created together
- You need to switch entire product families at runtime

Example: Java's `Integer.valueOf()` is a factory method; UI toolkits creating platform-specific buttons/menus use Abstract Factory.

TL;DR: Factory Method = one product via inheritance; Abstract Factory = product families via composition.

Keyword/Key mappings: Object creation | Interface | Subclass responsibility | Product families | Inheritance vs Composition

Q5: Explain the Observer Pattern and provide a real-world scenario.

Answer: The Observer pattern establishes a one-to-many relationship between objects such that one object (subject) keeps a list of its dependents (observers) and informs them of changes to state. Observers are updated automatically whenever the subject's state changes.

Polished Answer: Observer implements a publish-subscribe model with three key components:

1. **Subject:** Maintains observer list, provides attach/detach/notify methods
2. **Observer Interface:** Defines update() method
3. **Concrete Observers:** Implement update() to react to state changes

Real-world examples:

- Social media notifications (followers get updates)
- Stock price tracking systems
- Event handling in AWT/Swing (Java UI frameworks)
- Model-View-Controller (MVC) architecture

Benefits: Loose coupling between subject and observers, dynamic addition/removal of observers.

TL;DR: One-to-many dependency; subject notifies all observers when state changes. Used in event systems, MVC, notifications.

Keyword/Key mappings: Publisher-Subscriber | One-to-many | Subject/Observer | Event handling | Loose coupling | Notification

Q6: What is the Strategy Pattern and how does it support the Open/Closed Principle?

Answer: The Strategy Pattern enables a class to alter its behavior (or algorithm) at runtime. It defines a family of algorithms, encapsulates each one, and makes them interchangeable.

Polished Answer: Strategy uses composition to switch between algorithms without modifying context code. Components:

- **Context:** Holds strategy reference and delegates behavior
- **Strategy Interface:** Defines algorithm contract
- **Concrete Strategies:** Implement specific algorithms

Open/Closed Principle support: New strategies can be added as new classes without modifying existing context or strategies. The context remains closed for modification but open for extension.

Example: Payment systems supporting multiple payment methods (card, UPI, crypto), sorting algorithms (bubble sort, quick sort), compression algorithms.

TL;DR: Encapsulate interchangeable algorithms; switch at runtime. New strategies = new classes, no context changes.

Keyword/Key mappings: Runtime behavior switching | Algorithm family | Composition | OCP | Context delegation | Payment methods

Q7: What is the Builder Pattern and what problem does it solve?

Answer: The Builder Pattern solves the problem of creating complex objects with many optional parameters, where constructors become difficult to manage. It prevents constructor telescoping and supports step-by-step object creation.

Polished Answer: Builder separates construction from representation. Key features:

- **Step-by-step construction:** Set only required fields
- **Fluent API:** Chained methods (e.g., `.age(30).address("...")`)
- **Immutable objects:** No setters after construction
- **No constructor telescoping:** Avoids multiple overloaded constructors

Implementation steps:

1. Static nested Builder class
2. Public constructor with required parameters
3. Methods for optional parameters (return this)
4. `build()` method returns final object

Example: Creating User objects with optional fields like phone, address, nationality; `StringBuilder` in Java.

TL;DR: Build complex objects step-by-step with fluent API. Avoids long constructor parameter lists.

Keyword/Key mappings: Complex object creation | Fluent interface | Optional parameters | Immutability | Step-by-step construction | Constructor telescoping

Q8: Explain the Adapter Pattern vs Bridge Pattern. How are they different?

Answer: The Adapter Pattern converts one interface into another, allowing incompatible classes to work together. The Bridge Pattern separates an abstraction from its implementation so both can evolve independently.

Polished Answer:

Adapter Pattern:

- **Purpose:** Make incompatible interfaces compatible
- **When to use:** Integrating legacy code, third-party libraries
- **Structure:** Wraps Adaptee with Adapter implementing Target interface
- **Example:** USB-to-Ethernet adapter, MediaAdapter playing different audio formats

Bridge Pattern:

- **Purpose:** Decouple abstraction from implementation for independent evolution
- **When to use:** Both abstraction and implementation will change frequently
- **Structure:** Abstraction holds reference to Implementer interface
- **Example:** Remote control (abstraction) controlling TV/Radio devices (implementations)

Key difference: Adapter is retrospective (fixing incompatibility), Bridge is prospective (designing for flexibility).

TL;DR: Adapter = make incompatible things work together; Bridge = separate abstraction from implementation for independent changes.

Keyword/Key mappings: Interface compatibility | Legacy integration | Abstraction/Implementation separation | Structural patterns | Wrapper vs Decoupler

Q9: What is the Decorator Pattern and how does it differ from Proxy Pattern?

Answer: The Decorator Pattern adds new behavior to an object dynamically without changing its structure. It extends functionality by wrapping objects with decorator classes.

The Proxy Pattern controls access to an object.

Polished Answer:

Decorator Pattern:

- **Purpose:** Add responsibilities dynamically at runtime
- **Focus:** Extending functionality
- **Example:** Text editor adding bold/italic/spell-check features; Java's `BufferedReader` wrapping `FileReader`

Proxy Pattern:

- **Purpose:** Control access to an object
- **Focus:** Access control, lazy loading, security, remote access
- **Example:** Permission checking before file access, lazy-loaded database connections

Key differences:

- Decorator adds behavior; Proxy controls access
- Decorator can be stacked (multiple decorators); Proxy typically single layer
- Decorator client knows it's getting enhanced object; Proxy client may be unaware

TL;DR: Decorator = add features; Proxy = control access. Both wrap objects but with different intents.

Keyword/Key mappings: Runtime behavior addition | Wrapper pattern | Access control | Lazy loading | `BufferedReader` | Enhanced functionality

Q10: What are the SOLID Principles? Explain each briefly.

Answer: SOLID principles are five design principles for writing clean, maintainable, and scalable code. They were introduced by Robert C. Martin in his paper "Design Principles and Design Patterns" (2000).

Polished Answer:

S - Single Responsibility Principle (SRP): A class should have only one reason to change. Focus on one task.

O - Open/Closed Principle (OCP): Open for extension, closed for modification. Add new features via new classes, not by changing existing code.

L - Liskov Substitution Principle (LSP): Subclass objects should be replaceable for parent

objects without breaking program correctness.

I - Interface Segregation Principle (ISP): Clients shouldn't depend on interfaces they don't use. Create specific interfaces instead of one general interface.

D - Dependency Inversion Principle (DIP): Depend on abstractions, not concrete implementations. High-level modules should not depend on low-level modules.

Relationship with Design Patterns: Strategy, Factory, and Dependency Injection patterns help implement these principles.

TL;DR: SRP = one responsibility; OCP = extend don't modify; LSP = subtypes work; ISP = small interfaces; DIP = depend on abstractions.

Keyword/Key mappings: Object-oriented principles | Robert C. Martin | Code quality | Maintainability | Design guidelines | Loose coupling

TOP 25 MOST IMPORTANT QUESTIONS (11-25)

Q11: What is the Command Pattern and how does it support undo/redo?

Answer: The Command Pattern encapsulates a request as an object, allowing it to be stored, executed, and reversed later. It supports undo/redo by storing command objects in history stacks.

Polished Answer: Command transforms requests into standalone objects containing all request details. Components:

- **Command Interface:** Declares execute() method
- **Concrete Commands:** Implement execute() and optionally undo()
- **Invoker:** Triggers commands without knowing implementation
- **Receiver:** Actual object performing the action

Undo/redo implementation: Commands are stored in a stack. To undo, pop the last command and call undo(); to redo, push back to a redo stack.

Use cases: Remote controls, text editors (typing/deleting as commands), task scheduling, UI button actions.

TL;DR: Encapsulate requests as objects. Store in stacks for undo/redo. Decouples invoker

from receiver.

Keyword/Key mappings: Request encapsulation | Undo/Redo | Command history | Invoker/Receiver | Data-driven | UI actions

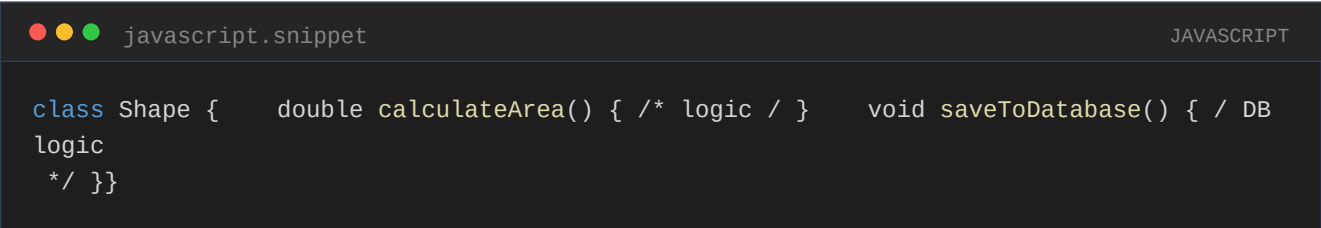
Q12: Explain the Single Responsibility Principle with an example.

Answer: The Single Responsibility Principle (SRP) states that a class should have only one responsibility and only one reason to change. This makes code easier to understand, maintain, and modify.

Polished Answer: SRP ensures each class focuses on a single functionality. If a class has multiple responsibilities, changes to one responsibility might affect others.

Example without SRP: A class that calculates area AND saves to database:

java

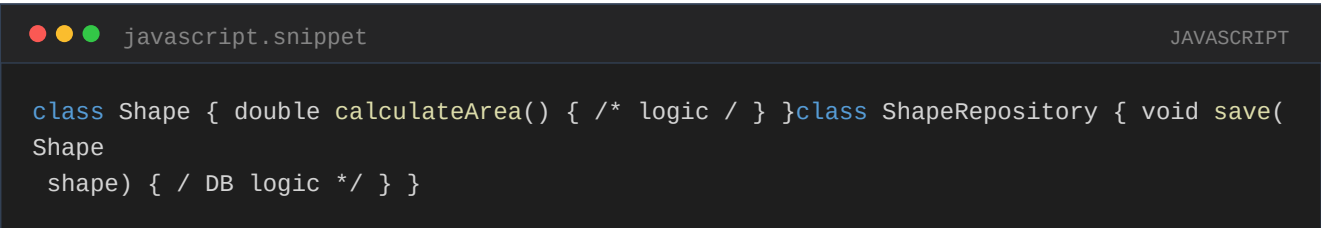


```
javascript.snippet JAVASCRIPT

class Shape {
  double calculateArea() { /* logic / }
  void saveToDatabase() { / DB logic */ }
}
```

With SRP: Separate classes:

java



```
javascript.snippet JAVASCRIPT

class Shape {
  double calculateArea() { /* logic / }
}

class ShapeRepository {
  void save(Shape shape) { / DB logic */ }
}
```

Significance: Modularity, maintainability, reduced impact of changes, better readability.

TL;DR: One class = one job = one reason to change. Separates concerns for easier maintenance.

Keyword/Key mappings: Modularity | Maintainability | Separation of concerns | Change impact | Readability | Cohesion

Q13: What is the Open/Closed Principle and how do design patterns enforce it?

Answer: The Open/Closed Principle (OCP) states that software entities should be open for extension but closed for modification. New functionality is added without changing existing code.

Polished Answer: OCP allows adding new features by extending existing classes rather than modifying them. Design patterns implementing OCP:

- **Strategy Pattern:** Add new strategies as new classes
- **Decorator Pattern:** Add behaviors by wrapping objects
- **Factory Pattern:** Add new product types without changing client code
- **Observer Pattern:** Add new observers without modifying subject

Example: A payment system can add UPI or crypto payments by creating new strategy classes without changing checkout logic.

TL;DR: Extend by adding new code, not modifying existing code. Patterns provide the structure for this.

Keyword/Key mappings: Extension vs Modification | New classes | Strategy/Decorator/Factory | Flexibility | Backward compatibility

Q14: How does the Dependency Inversion Principle facilitate loose coupling?

Answer: The Dependency Inversion Principle (DIP) states that high-level modules should depend on abstractions, not concrete implementations. This reduces coupling and makes the system more flexible.

Polished Answer: DIP inverts the traditional dependency flow:

- **Traditional:** High-level Low-level (tight coupling)
- **With DIP:** High-level Abstraction Low-level (loose coupling)

Implementation: Use interfaces/abstract classes as contracts. High-level modules depend on these abstractions; low-level modules implement them.

Design patterns implementing DIP:

- **Factory Method:** Client depends on product interface
- **Dependency Injection:** Container injects dependencies
- **Strategy Pattern:** Context depends on strategy interface

Example: Instead of class A directly creating B (`new B()`), both depend on interface IB, injected through constructor.

TL;DR: Depend on abstractions, not implementations. Inverts dependency flow for flexibility.

Keyword/Key mappings: Abstraction dependency | Interface contracts | High/Low-level modules | DI | Inversion of control | Flexibility

Q15: Explain the Factory Pattern vs Abstract Factory with examples.

Answer: Factory Method defines an interface for creating objects, allowing subclasses to alter the type created. Abstract Factory creates families of related objects without specifying concrete classes.

Polished Answer:

Factory Method:

- Creates ONE product through inheritance
- Creator subclasses override `factoryMethod()`
- Example: `ShapeFactory.createShape("circle")` returns Circle object

Abstract Factory:

- Creates FAMILIES of related products
- Uses composition with factory objects
- Example: UI toolkit creating `WindowsButton` + `WindowsMenu` together, or `MacButton` + `MacMenu`

When to use Factory Method: Single product with varying types
When to use Abstract Factory: Multiple related products that must be compatible

TL;DR: Factory = one product type; Abstract Factory = family of compatible products.

Keyword/Key mappings: Object creation | Single vs Multiple products | Inheritance vs Composition | UI toolkits | Product families

Q16: What is the Facade Pattern and when should it be used?

Answer: The Facade Pattern provides a single, unified interface that hides the complexity of multiple interacting subsystems. It reduces coupling between clients and subsystem classes.

Polished Answer: Facade acts as a simplified gateway to complex systems:

- **Purpose:** Simplify subsystem interaction
- **Benefits:** Reduced coupling, easier usage, cleaner client code
- **Example:** Home theater system with `watchMovie()` method coordinating projector, sound system, and lights

When to use:

- Complex subsystem with many dependencies
- Need to layer subsystems (facade per layer)
- Want to decouple client from subsystem internals

TL;DR: Single entry point to complex subsystem. Simplifies client interactions.

Keyword/Key mappings: Unified interface | Complexity hiding | Decoupling | Home theater | Subsystem gateway | Simplified API

Q17: How does the Prototype Pattern differ from object cloning using `clone()` in Java?

Answer: The Prototype Pattern focuses on creating new objects by copying existing ones (prototypes), while Java's `clone()` is a low-level mechanism for object copying.

Polished Answer:

Prototype Pattern:

- Provides controlled, explicit cloning logic
- Hides cloning complexity behind an interface
- Supports deep copying of complex object graphs
- Centralized `clone()` method (usually in Abstract class)

Java `clone()`:

- Low-level object copying mechanism
- Requires implementing `Cloneable` interface

- Default is shallow copy (shares internal objects)
- Can cause issues with mutable fields

Example: Game enemy objects copied from prototype with proper deep copy of weapons, health state.

TL;DR: Prototype = controlled cloning pattern; clone() = low-level Java mechanism. Prototype ensures proper deep copying.

Keyword/Key mappings: Object copying | Deep vs Shallow copy | Cloneable interface | Game objects | Cloning logic encapsulation

Q18: What is Inversion of Control (IoC)?

Answer: Inversion of Control (IoC) is a design principle where object creation and management are handled by a container or framework instead of the application itself. This reduces coupling and improves flexibility.

Polished Answer: IoC shifts control of dependencies from the class to an external entity:

- **Traditional:** Class creates its dependencies (new B())
- **With IoC:** Framework injects dependencies (constructor injection)
- **Implementation:** Dependency Injection (DI) is the most common IoC pattern
- **Example:** Spring container creates and injects objects

Benefits: Loose coupling, easier testing, centralized configuration.

TL;DR: Framework manages object creation/lifecycle. DI is the most common IoC implementation.

Keyword/Key mappings: Dependency Injection | Control inversion | Framework managed | Spring | Decoupling | Object lifecycle

Q19: What is the Chain of Responsibility Pattern?

Answer: The Chain of Responsibility Pattern passes requests through a chain of handlers. Each handler decides whether to process the request or pass it to the next handler in the chain.

Polished Answer: Components:

- **Client:** Originates request

- **Handler:** Interface/class that defines handling contract
- **Concrete Handlers:** Sequential request processors

Key benefits:

- Loose coupling between sender and receiver
- Multiple objects can handle request
- Dynamic handler chain modification

Example: Approval system where request passes through manager director CEO until approved.

When to use: When multiple objects may handle a request, when handler is determined at runtime, when you want to avoid explicit handler specification.

TL;DR: Request passes through sequential handlers until one processes it. Loose coupling, flexible handling.

Keyword/Key mappings: Handler chain | Sequential processing | Request passing | Approval workflow | Dynamic handlers | Loose coupling

Q20: How can you achieve thread-safe Singleton patterns in Java?

Answer: A thread-safe Singleton class ensures object initialization in the presence of multiple threads. It can be achieved using multiple approaches.

Polished Answer:

1. Using Enums (Simplest):

java

```

javascript.snippet
JAVASCRIPT

public enum ThreadSafeSingleton {
    SINGLETON_INSTANCE;
    public void display() { /*
    logic
    */ }}

```

Java inherently provides synchronization for enums.

2. Static Field Initialization (Eager):

java

```
javascript.snippet JAVASCRIPT

public class ThreadSafeSingleton {    private static final ThreadSafeSingleton INSTANCE

= new ThreadSafeSingleton();    private ThreadSafeSingleton() { }    public static
ThreadSafeSingleton
getInstance() { return INSTANCE; }}
```

Created at class loading; thread-safe but not lazy.

3. Synchronized Method (Lazy but slow):

java

```
javascript.snippet JAVASCRIPT

synchronized public static ThreadSafeSingleton getInstance() { /* logic */ }
```

Performance impact in multi-threaded environments.

4. Double-Checked Locking (Best):

java

```
javascript.snippet JAVASCRIPT

public static ThreadSafeSingleton getInstance() {    if (instance == null) {
synchronized
(ThreadSafeSingleton.class) {                if (instance == null) {
instance
= new ThreadSafeSingleton();                }            }    }    return instance;}
```

Lazy initialization + thread safety + good performance.

TL;DR: Enums (simplest), static field (eager), synchronized (slow), double-checked locking (best balance).

Keyword/Key mappings: Thread safety | Enum | Double-checked locking | Lazy initialization
| Synchronization | Multi-threading

Q21: Explain the Bridge Pattern with a real-world scenario.

Answer: The Bridge Pattern separates an abstraction from its implementation so both can evolve independently. It splits large classes into two hierarchies: abstraction and implementation.

Polished Answer: Components:

- **Abstraction:** Core of the pattern, holds reference to Implementer
- **Refined Abstraction:** Extends abstraction with details
- **Implementer:** Interface for implementation classes
- **Concrete Implementation:** Implements Implementer interface

Real-world example: Remote control (abstraction) controlling different devices (implementations):

- Abstraction: RemoteControl with basic buttons
- Refined: AdvancedRemote with mute, channel presets
- Implementer: Device interface (on/off/volume)
- Concrete: TV, Radio, Projector

Benefits: Both hierarchies can change independently; client code depends only on abstraction.

TL;DR: Separate what from how. Abstraction and implementation evolve independently.

Keyword/Key mappings: Abstraction/Implementation separation | Remote control | Device interface | Independent evolution | Two hierarchies

Q22: What is the Proxy Pattern and what are its types?

Answer: The Proxy Pattern represents the functionality of other classes. It provides a substitute or placeholder for another object to control access.

Polished Answer: Proxy creates an intermediary between client and real service:

- **Components:** ServiceInterface, Service (real object), Proxy (intermediary), Client
- **Proxy capabilities:** Lazy initialization, logging, caching, access control

Types of proxies:

- **Remote Proxy:** Represents object in different location
- **Virtual Proxy:** Lazy loading of heavy objects
- **Protection Proxy:** Access control based on permissions
- **Smart Proxy:** Adds logging, caching, reference counting

Example: Permission check before file access; lazy-loaded database connections.

TL;DR: Intermediary controlling access to real object. Types: remote, virtual, protection, smart.

Keyword/Key mappings: Placeholder | Access control | Lazy loading | Remote object | Caching | Intermediary

Q23: What is the Composite Pattern and when is it useful?

Answer: The Composite Pattern composes objects into tree structures to represent part-whole hierarchies. Clients treat individual objects and compositions uniformly.

Polished Answer: Key components:

- **Component:** Interface common to all objects
- **Leaf:** Individual object (no children)
- **Composite:** Container holding children

When to use:

- Represent part-whole hierarchies
- Clients should treat single and composite objects uniformly
- Tree-like structures (file systems, organization charts, UI components)

Example: File system where File (leaf) and Directory (composite) both implement FileSystemComponent interface.

TL;DR: Tree structure of objects; treat individual and group of objects identically. Useful for hierarchies.

Keyword/Key mappings: Part-whole hierarchy | Tree structure | Leaf/Composite | File system | Uniform treatment | Recursion

Q24: What is the Iterator Pattern and how does it support encapsulation?

Answer: The Iterator Pattern allows clients to traverse a collection without exposing its internal structure. It separates traversal logic from the collection.

Polished Answer: Benefits:

- **Encapsulation:** Collection implementation hidden from client
- **Multiple traversals:** Different iterators can traverse same collection differently
- **Uniform interface:** Same iteration logic for different collections

Example: Custom data structure with iterator allowing loop-through without knowing internal

storage (array, list, tree).

Java implementation: Iterator interface with `hasNext()`, `next()`, `remove()` methods.

TL;DR: Traverse collection without exposing internals. Uniform iteration interface.

Keyword/Key mappings: Traversal | Encapsulation | Collection access | Uniform interface | `hasNext/next` | Abstraction

Q25: What is the State Pattern and how does it differ from Strategy Pattern?

Answer: The State Pattern changes an object's behavior based on its internal state. The Strategy Pattern changes behavior by swapping algorithms externally.

Polished Answer:

State Pattern:

- State transitions managed INSIDE the context
- Behavior changes automatically based on current state
- Multiple states encapsulated as classes
- Example: Vending machine states (no-coin, has-coin, sold)

Strategy Pattern:

- Strategy selection done by CLIENT
- Algorithms swapped externally
- Single behavior per context
- Example: Payment methods (client chooses card/UPI)

Key difference: State = automatic internal changes; Strategy = external algorithm selection.

TL;DR: State = behavior changes with internal state; Strategy = client chooses algorithm externally.

Keyword/Key mappings: Internal state | Automatic transition | Vending machine | External selection | Algorithm swapping | Behavior change

TOP 50 MOST IMPORTANT QUESTIONS (26-50)

Q26: What is the Template Method Pattern?

Answer: The Template Method Pattern defines the skeleton of an algorithm in a base class, allowing subclasses to override specific steps without changing the algorithm's structure.

Polished Answer: Key features:

- **Base class:** Defines algorithm skeleton with fixed sequence
- **Abstract methods:** Steps that subclasses implement
- **Hook methods:** Optional methods with default behavior
- **final template method:** Prevents algorithm structure modification

Example: Data processing flow where steps are fixed (read process save) but processing step varies by subclass.

TL;DR: Fixed algorithm skeleton; subclasses vary specific steps. Uses inheritance.

Keyword/Key mappings: Algorithm skeleton | Base class | Hook methods | Inheritance | Fixed sequence | Customizable steps

Q27: When is the Visitor Pattern useful despite its complexity?

Answer: The Visitor Pattern is useful when adding new operations to a stable object structure without modifying existing classes.

Polished Answer: When to use:

- Object structure rarely changes but operations change frequently
- Related operations need to be grouped together
- Different operations on heterogeneous objects

Example: Compiler operations (type checking, code generation) on syntax tree nodes.

Trade-off: Complex to implement, but adding new operations becomes easier without modifying node classes.

TL;DR: Add operations without changing object structure. Useful when structure is stable but operations change often.

Keyword/Key mappings: Stable structure | New operations | Double dispatch | Compiler | Operation grouping | Extensibility

Q28: What is the Flyweight Pattern and when is it most effective?

Answer: The Flyweight Pattern handles large numbers of similar objects efficiently by sharing common state (intrinsic) and separating variable state (extrinsic).

Polished Answer: Key concepts:

- **Intrinsic state:** Shared, immutable data (e.g., font, style)
- **Extrinsic state:** Varies per object (e.g., position, content)
- **Flyweight factory:** Manages shared objects

When effective:

- Large number of similar objects
- Memory usage must be minimized
- Most object state can be externalized

Example: Text editor where characters share font/style objects; position and content stored separately.

TL;DR: Share common state across many objects. Minimize memory usage for similar objects.

Keyword/Key mappings: Object sharing | Memory optimization | Intrinsic/Extrinsic state | Text editor | Object pooling | Fine-grained objects

Q29: What is the Mediator Pattern and how does it reduce coupling?

Answer: The Mediator Pattern reduces coupling by centralizing communication logic between objects into a single mediator object.

Polished Answer: Key aspects:

- Objects interact through mediator instead of direct references
- Changes in communication affect only mediator
- Centralizes complex communication logic

Example: Chat application where users send messages through a chat mediator; air traffic control coordinating aircraft.

Benefits: Reduced coupling, simplified communication, easier maintenance.

TL;DR: Central communication hub. Objects don't reference each other directly.

Keyword/Key mappings: Centralized communication | Coupling reduction | Chat application
| Air traffic control | Communication logic | Mediator object

Q30: What is the Memento Pattern and when is it preferred?

Answer: The Memento Pattern captures and restores an object's internal state without exposing its internal details.

Polished Answer: Key components:

- **Originator:** Object whose state needs saving
- **Memento:** Stores state snapshot
- **Caretaker:** Manages memento lifecycle

When preferred:

- State rollback/undo functionality needed
- Encapsulation must be preserved
- Snapshot of complex state required

Example: Text editor saving document state for undo functionality.

TL;DR: Save and restore state without breaking encapsulation. Used for undo functionality.

Keyword/Key mappings: State snapshot | Undo/Redo | Encapsulation |
Originator/Memento/Caretaker | Text editor | State rollback

Q31: What is the Null Object Pattern?

Answer: The Null Object Pattern replaces null references with a non-functional object that implements the same interface, avoiding null checks.

Polished Answer: Benefits:

- Eliminates repetitive null checks
- Safer and more readable code
- Provides default behavior when data unavailable

Example: Instead of checking `if (logger != null)`, use a `NullLogger` that does nothing when `log()` is called.

TL;DR: Replace null with "do nothing" object. Avoids null checks, provides default behavior.

Keyword/Key mappings: Null replacement | Default behavior | Safety | Readability | Do-nothing object | Null checks elimination

Q32: What is the difference between static factory methods and Factory Pattern?

Answer: Static factory methods are simple methods returning objects, while Factory Pattern is a structured design approach using abstraction for object creation.

Polished Answer:

Static Factory Methods:

- Tied to a single class
- No polymorphism support
- Simple implementation
- Example: `createUser()` method

Factory Pattern:

- Uses interfaces and subclasses
- Supports extensibility and polymorphism
- More structured approach
- Example: Factory class with method overridden by subclasses

TL;DR: Static factory = simple method; Factory Pattern = structured abstraction with polymorphism.

Keyword/Key mappings: Static vs Polymorphic | Single class vs Interface | Extensibility | Simple vs Structured | Object creation

Q33: How does Lazy Initialization work in Singleton Pattern?

Answer: Lazy Initialization delays object creation until it is actually needed, instead of creating it at class loading time.

Polished Answer: Key aspects:

- Instance created only when `getInstance()` first called
- Saves memory and startup time

- Useful for heavy objects

Example: Logging service created only when application first logs a message.

Trade-off: Slightly more complex than eager initialization but better resource management.

TL;DR: Create object on first use, not at class loading. Saves resources for heavy objects.

Keyword/Key mappings: Delayed creation | On-demand initialization | Resource efficiency | getInstance() | Heavy objects | Startup optimization

Q34: What is the difference between Composition and Inheritance in design patterns?

Answer: Composition builds behavior by combining objects, while inheritance relies on extending classes to reuse behavior.

Polished Answer:

Composition:

- Better flexibility, avoids tight coupling
- Behavior can be changed at runtime
- Promoted by Strategy, Decorator patterns
- "Has-a" relationship

Inheritance:

- Creates rigid hierarchies
- Behavior fixed at compile time
- "Is-a" relationship
- Can break with changing requirements

Design patterns: Strategy uses composition; Template Method uses inheritance.

TL;DR: Composition = "has-a", flexible; Inheritance = "is-a", rigid. Patterns often prefer composition.

Keyword/Key mappings: Has-a vs Is-a | Flexibility | Runtime behavior | Strategy/Decorator | Tight coupling | Code reuse

Q35: What role does encapsulation play in behavioral design patterns?

Answer: Encapsulation helps behavioral design patterns by hiding implementation details and exposing only what clients need.

Polished Answer: Benefits:

- Separates what an object does from how it does it
- Behavior changes without affecting client code
- Implementation details hidden behind interfaces

Example: Command Pattern wraps requests in objects; caller doesn't know execution details. Strategy Pattern encapsulates algorithms.

TL;DR: Hide "how", expose "what". Enables behavior change without client impact.

Keyword/Key mappings: Information hiding | Interface exposure | Command Pattern | Strategy Pattern | Client isolation | Behavior abstraction

Q36: What is a UML diagram and its purpose in displaying design patterns?

Answer: A UML (Unified Modeling Language) diagram visually represents the structure and relationships of classes in a design pattern.

Polished Answer: Purpose:

- Visualizes classes, interfaces, relationships
- Makes design easier to understand and communicate
- Documents pattern structure for implementation

Example: Observer Pattern UML shows Subject, Observer interface, and concrete observers with their relationships.

TL;DR: Visual representation of pattern structure. Aids understanding and communication.

Keyword/Key mappings: Visual modeling | Class relationships | Pattern documentation | Communication tool | Structure representation | Design visualization

Q37: How can Design Patterns assist in refactoring existing code?

Answer: Design patterns help improve existing code by providing proven solutions to recurring design problems without changing behavior.

Polished Answer: Refactoring benefits:

- Improve code structure and maintainability
- Reduce code duplication
- Make system easier to extend
- Apply patterns to problematic code sections

Example: Large conditional block refactored into Strategy Pattern, where each condition becomes a separate strategy class.

TL;DR: Patterns provide target solutions for refactoring. Improve structure without changing behavior.

Keyword/Key mappings: Code improvement | Structure enhancement | Duplication reduction | Extensibility | Refactoring targets | Proven solutions

Q38: When should you avoid using design patterns?

Answer: Design patterns should be avoided when they add unnecessary complexity to a simple problem, leading to over-engineering.

Polished Answer: Avoid patterns when:

- Problem can be solved with simpler approach
- Pattern introduces unnecessary abstraction
- No clear design problem to solve
- Future flexibility not needed

Example: Using Strategy Pattern for single tax calculation rule adds unnecessary classes; simple method suffices.

TL;DR: Use patterns only when needed. Simplicity first; patterns for genuine complexity.

Keyword/Key mappings: Over-engineering | Simplicity | Unnecessary complexity | YAGNI | Simple solutions first | Practical judgment

Q39: How do design patterns help in managing dependencies in large-scale applications?

Answer: Design patterns manage dependencies by structuring interactions through abstractions instead of direct class-to-class references.

Polished Answer: Benefits:

- Reduce tight coupling using interfaces
- Make dependency changes localized
- Centralize dependency management
- Improve testability

Example: Factory and Dependency Injection patterns manage object creation without spreading dependency logic across codebase.

TL;DR: Patterns provide abstraction layers for dependency management. Reduce coupling in large systems.

Keyword/Key mappings: Dependency management | Abstraction layers | Loose coupling | Factory/DI | Microservices | Testability

Q40: How does the Command Pattern work in UI frameworks?

Answer: The Command Pattern encapsulates user actions as command objects, allowing execution, queuing, or undoing independently of UI components.

Polished Answer: UI framework applications:

- Button clicks, menu actions, keyboard shortcuts as commands
- Undo/redo functionality
- Command history tracking
- Task scheduling

Example: Every user action wrapped as command object; saved commands replayed for undo.

TL;DR: UI actions as command objects. Enables undo/redo, history, and decoupled execution.

Keyword/Key mappings: UI actions | Command objects | Undo/Redo | Event handling | Action encapsulation | Framework design

Q41: What are anti-patterns and how do they differ from design patterns?

Answer: Anti-patterns are common solutions to problems that are ineffective or harmful,

while design patterns are proven best practices.

Polished Answer:

Anti-patterns:

- Describe what NOT to do
- Have negative consequences
- Example: God Object (one class does too much), Spaghetti Code

Design Patterns:

- Proven effective solutions
 - Promote best practices
 - Example: Facade, Strategy distribute responsibilities properly
- TL;DR:** Anti-patterns = harmful solutions to avoid; Design patterns = proven solutions to adopt.

Keyword/Key mappings: Bad practices | God Object | Spaghetti Code | Negative consequences | What not to do | Contrast

Q42: Can multiple design patterns be combined in a single solution?

Answer: Yes, multiple design patterns are often combined to solve complex design problems more effectively.

Polished Answer: Patterns complement each other:

- Different patterns address different concerns
- Improves flexibility, scalability, maintainability
- Common in real-world applications

Example: MVC architecture combines Observer (view updates), Strategy (business logic), Factory (object creation).

TL;DR: Patterns can and should be combined. Different concerns, different patterns.

Keyword/Key mappings: Pattern combination | MVC | Complementary patterns | Complex solutions | Architecture design | Synergy

Q43: What factors should be considered before choosing a design pattern?

Answer: Choosing a design pattern requires understanding problem context and long-term impact on the system.

Polished Answer: Consider:

- Nature of the problem
- Complexity and change frequency
- Impact on flexibility and performance
- Maintainability requirements
- Team familiarity with pattern

Example: Singleton seems simple for configuration, but Dependency Injection might be better for testing.

TL;DR: Match pattern to problem, not vice versa. Consider context, change, and maintenance.

Keyword/Key mappings: Problem analysis | Context | Flexibility | Performance | Maintainability | Pattern selection

Q44: How do design patterns improve testability of code?

Answer: Design patterns improve testability by promoting loose coupling and separation of responsibilities.

Polished Answer: Benefits:

- Dependencies easily mocked or replaced
- Behavior isolated into smaller components
- Clear interfaces for testing
- Support unit testing

Example: Strategy Pattern allows testing different strategies independently with mock objects.

TL;DR: Loose coupling + clear interfaces = easier testing. Patterns enable mocking and isolation.

Keyword/Key mappings: Unit testing | Mocking | Loose coupling | Isolation | Test interfaces | Dependency injection

Q45: How do design patterns evolve with changing software requirements?

Answer: Design patterns evolve by being adapted, combined, or replaced as requirements and constraints change over time.

Polished Answer: Evolution aspects:

- Patterns refactored or composed for new requirements
- Some patterns become unnecessary as frameworks evolve
- New patterns emerge for new problem types

Example: Early Singleton for shared resources evolves to Dependency Injection for scalability and testability.

TL;DR: Patterns adapt or replace as needs change. Framework evolution impacts pattern relevance.

Keyword/Key mappings: Adaptation | Pattern replacement | Framework evolution | Requirement changes | Pattern relevance | Refactoring

Q46: What design patterns are used in Java's JDK library?

Answer: Several design patterns are implemented in Java's JDK library.

Polished Answer: Examples:

- **Decorator:** `BufferedReader`, `BufferedWriter` wrap readers/writers
- **Singleton:** `Runtime`, `Calendar` classes
- **Factory:** `Integer.valueOf()`, `Calendar.getInstance()`
- **Observer:** AWT/Swing event handling
- **Iterator:** Collection iterators
- **Adapter:** `Arrays.asList()` adapting arrays

TL;DR: JDK uses Decorator, Singleton, Factory, Observer, Iterator, Adapter patterns extensively.

Keyword/Key mappings: JDK implementations | Java library | `BufferedReader` | `Runtime` | Event handling | Collections

Q47: What is the Gang of Four (GoF)?

Answer: The Gang of Four (GoF) refers to the four authors who introduced design patterns in their book "Design Patterns: Elements of Reusable Object-Oriented Software" (1995).

Polished Answer: The four authors:

- Erich Gamma
- Richard Helm
- Ralph Johnson
- John Vlissides

They documented 23 classic design patterns in object-oriented software development.

TL;DR: Four authors who created the design pattern catalog. Wrote the foundational book in 1995.

Keyword/Key mappings: Authors | 1995 book | 23 patterns | Object-oriented design | Foundational work | Pattern catalog

Q48: What is the difference between Design Patterns and Algorithms?

Answer: Algorithms give step-by-step solutions for specific tasks, while design patterns provide general guidelines for organizing software.

Polished Answer:

Algorithms:

- Solve computational problems
- Step-by-step procedure
- Focus on exact computation
- Example: Sorting algorithm

Design Patterns:

- Solve design problems
- General blueprints for organization
- Focus on architecture and object interactions
- Example: Factory pattern

TL;DR: Algorithms = computational steps; Design patterns = architectural blueprints.

Keyword/Key mappings: Computation vs Architecture | Step-by-step vs Guidelines | Problem solving | Object interaction | Blueprint vs Procedure

Q49: How does encapsulation help in the Iterator Pattern?

Answer: The Iterator Pattern supports encapsulation by allowing clients to traverse collections without exposing internal structure.

Polished Answer: Benefits:

- Collection implementation hidden from client
- Traversal logic separated from collection
- Uniform traversal interface regardless of internal storage

Example: Custom data structure with iterator allowing loop-through without knowing if data stored in array, list, or tree.

TL;DR: Traverse without exposing internals. Uniform interface for different data structures.

Keyword/Key mappings: Traversal abstraction | Data structure hiding | Uniform interface | Array/List/Tree | Collection iteration | Encapsulation

Q50: What is the Model-View-Controller (MVC) design pattern?

Answer: MVC is a design pattern for separating application concerns into Model, View, and Controller components.

Polished Answer: Components:

- **Model:** Represents data and business logic
- **View:** Visualizes model data
- **Controller:** Handles input, updates model, refreshes view

Flow: Client request → Controller → Model update → View render → Response

Benefits: Separation of concerns, maintainability, testability.

TL;DR: Separation of data (Model), presentation (View), and logic (Controller) for maintainable apps.

Keyword/Key mappings: Separation of concerns | Data/Presentation/Logic | Web

TOP 100 MOST IMPORTANT QUESTIONS (51-100)

Q51: How are Design Principles different from Design Patterns?

Answer: Design Principles are general guidelines for writing clean, maintainable code, while Design Patterns are proven, reusable solutions to specific problems.

Polished Answer:

Design Principles:

- General guidelines
- Focus on code quality, flexibility
- Examples: SOLID, DRY, KISS

Design Patterns:

- Specific solutions to recurring problems
- Standard approaches adaptable to scenarios
- Examples: Singleton, Factory, Observer

TL;DR: Principles = guidelines; Patterns = specific solutions.

Keyword/Key mappings: Guidelines vs Solutions | SOLID/DRY/KISS | Code quality | Specific vs General | Best practices | Adaptability

Q52: What is the Decorator Pattern and how is it applied in a codebase?

Answer: The Decorator Pattern extends functionality of existing objects dynamically without modifying their structure.

Polished Answer: Real-world example: Text editor adding spell-check, formatting, encryption features dynamically.

Implementation steps:

1. Create component interface
2. Create concrete component

3. Create abstract decorator implementing interface
4. Create concrete decorators extending abstract decorator
5. Decorate objects as needed

Example: PlainText wrapped with BoldTextDecorator, ItalicTextDecorator.

TL;DR: Add features at runtime by wrapping objects. No modification of original class needed.

Keyword/Key mappings: Runtime extension | Object wrapping | Text editor | Bold/Italic | Dynamic features | No class modification

Q53: What is the difference between Bridge Pattern and Adapter Pattern?

Answer: Bridge separates abstraction from implementation for independent evolution; Adapter converts one interface to another for compatibility.

Polished Answer:

Bridge:

- Decouples abstraction from implementation
- Both can evolve independently
- Used prospectively (during design)
- Example: Remote control + device implementations

Adapter:

- Makes incompatible interfaces compatible
- Wraps existing class
- Used retrospectively (fixing compatibility)
- Example: USB-to-Ethernet adapter

TL;DR: Bridge = design for flexibility; Adapter = fix incompatibility.

Keyword/Key mappings: Abstraction/Implementation | Interface compatibility | Prospective vs Retrospective | Remote control | Legacy code | Design intent

Q54: What is the Command Pattern used for in task scheduling?

Answer: The Command Pattern encapsulates requests as objects, enabling task scheduling,

queuing, and execution management.

Polished Answer: Command Pattern for scheduling:

- Commands stored in queues
- Executed at specific times
- Can be serialized for distributed systems
- Supports retry mechanisms

Example: Job scheduling systems where tasks are command objects executed by scheduler.

TL;DR: Commands as queue objects for scheduling. Enables deferred execution and task management.

Keyword/Key mappings: Task queue | Deferred execution | Job scheduling | Command objects | Retry mechanism | Distributed systems

Q55: How does the Observer Pattern work in event handling frameworks?

Answer: Observer Pattern enables event-driven programming where objects notify interested parties about state changes.

Polished Answer: In event frameworks:

- Event sources are subjects
- Event listeners are observers
- Event firing triggers observer updates
- Decoupled event handling

Example: Java AWT/Swing where button clicks notify registered listeners; JavaScript DOM events.

TL;DR: Events trigger observer notifications. Foundation for event-driven programming.

Keyword/Key mappings: Event handling | AWT/Swing | DOM events | Event listeners | Notification | Event-driven architecture

Q56: What is the difference between Strategy Pattern and Template Method Pattern?

Answer: Strategy uses composition to swap algorithms; Template Method uses inheritance

to vary specific steps.

Polished Answer:

Strategy:

- Composition-based
- Runtime algorithm switching
- Client selects strategy
- Example: Payment methods

Template Method:

- Inheritance-based
- Algorithm skeleton fixed
- Subclasses vary steps
- Example: Data processing flow

TL;DR: Strategy = swap entire algorithm (composition); Template = vary steps (inheritance).

Keyword/Key mappings: Composition vs Inheritance | Algorithm swapping | Runtime selection | Fixed skeleton | Payment vs Processing | Design flexibility

Q57: What is the Prototype Pattern used for in game development?

Answer: Prototype Pattern creates objects by copying prototypes, useful in games for creating enemies, items, and other repeated elements.

Polished Answer: Game development uses:

- Enemy objects copied from prototype
- Deep copying of internal state (weapons, health)
- Performance optimization vs new keyword
- Variation through prototype modification

Example: Prototype enemy with default weapons; clones created and customized.

TL;DR: Clone prototypes for game objects. Better performance than new, enables deep copying.

Keyword/Key mappings: Game objects | Cloning | Deep copy | Performance | Enemy creation | Prototype modification

Q58: How does the Builder Pattern improve code readability?

Answer: Builder Pattern improves readability through fluent interfaces and step-by-step object construction.

Polished Answer: Readability benefits:

- Fluent API (`user.age(30).address("...")`)
- Named methods instead of positional parameters
- Self-documenting code
- Clear construction intent

Example: `new UserBuilder("John", "Doe").age(30).build()` vs `new User("John", "Doe", 30, null, null)`.

TL;DR: Fluent, readable object construction. No confusing parameter positions.

Keyword/Key mappings: Fluent API | Readability | Named methods | Self-documenting | Method chaining | Clear intent

Q59: What are the components of the Composite Entity Pattern?

Answer: Composite Entity Pattern is used in EJB persistence, representing object graphs with dependent objects.

Polished Answer: Components:

- **Composite Entity:** Primary entity bean
- **Coarse-Grained Object:** Contains dependent objects
- **Dependent Object:** Depends on coarse-grained object
- **Strategies:** Implementation approach

Purpose: Manage dependent object lifecycle with entity persistence.

TL;DR: EJB pattern for managing object graphs in persistence.

Keyword/Key mappings: EJB | Object graph | Persistence | Coarse-grained | Dependent objects | Entity beans

Q60: How do design patterns promote code reusability?

Answer: Design patterns promote reusability through proven, adaptable solutions that can be applied across projects.

Polished Answer: Reusability aspects:

- Proven solutions applicable in multiple contexts
- Standardized approaches reduce rework
- Template solutions adaptable per requirements
- Reduce redundant code

Example: Factory pattern reused across projects for object creation without exposing logic.

TL;DR: Patterns are reusable templates. Apply proven solutions instead of reinventing.

Keyword/Key mappings: Template solutions | Cross-project reuse | Standardized approaches | Reduced rework | Proven patterns | Adaptability

Q61: What is the difference between eager and lazy initialization in Singleton?

Answer: Eager initialization creates instance at class loading; lazy initialization creates instance on first use.

Polished Answer:

Eager:

- Instance created at class loading
- Simple, thread-safe
- Used when instance always needed
- Example: Static field initialization

Lazy:

- Instance created on first getInstance()
- Saves resources
- Requires synchronization for thread safety
- Example: Double-checked locking

TL;DR: Eager = create early; Lazy = create on demand.

Keyword/Key mappings: Initialization timing | Class loading vs First use | Resource management | Thread safety | Static field | On-demand creation

Q62: How does the Adapter Pattern help in integrating legacy code?

Answer: Adapter Pattern wraps legacy classes with new interfaces, enabling integration without modifying legacy code.

Polished Answer: Integration benefits:

- Legacy code untouched
- New interfaces exposed to clients
- Gradual migration possible
- Compatibility layer created

Example: Legacy payment system wrapped with modern API interface for new clients.

TL;DR: Wrap legacy code with modern interfaces. No legacy modification needed.

Keyword/Key mappings: Legacy integration | Interface wrapping | Migration | Compatibility | Modern API | Code preservation

Q63: What is the role of context in the Strategy Pattern?

Answer: The context in Strategy Pattern holds a reference to a strategy and delegates behavior to it.

Polished Answer: Context responsibilities:

- Holds current strategy reference
- Delegates algorithm execution to strategy
- Allows strategy change at runtime
- Abstracted from strategy implementation

Example: Checkout class (context) delegates payment processing to selected strategy (card, UPI).

TL;DR: Context delegates to strategy. Selects and uses without knowing implementation.

Keyword/Key mappings: Delegation | Strategy reference | Runtime change | Checkout context | Implementation abstraction | Behavior delegation

Q64: How does the Singleton Pattern create global state?

Answer: Singleton creates global state by providing a single, globally accessible instance shared across the application.

Polished Answer: Global state implications:

- Single instance accessible everywhere
- State shared across all clients
- Harder to isolate in tests
- Potential for unintended coupling

Example: Configuration singleton holding app-wide settings modified from anywhere.

TL;DR: One shared instance = global state. Convenient but creates coupling.

Keyword/Key mappings: Global access | Shared state | Test isolation | Coupling | Configuration | App-wide instance

Q65: What is the difference between Object Pool and Flyweight Pattern?

Answer: Object Pool reuses objects to reduce creation cost; Flyweight shares objects to reduce memory usage.

Polished Answer:

Object Pool:

- Reuses existing objects
- Focus on expensive creation
- Objects return to pool after use
- Example: Database connection pool

Flyweight:

- Shares intrinsic state across objects
- Focus on memory efficiency
- Extrinsic state varies per use
- Example: Character glyphs in text editor

TL;DR: Pool = reuse objects; Flyweight = share state.

Keyword/Key mappings: Reuse vs Share | Object creation cost | Memory efficiency | Connection pool | Shared state | Resource management

Q66: How does the Visitor Pattern use double dispatch?

Answer: Visitor Pattern uses double dispatch to execute type-specific operations by combining element type and visitor type.

Polished Answer: Double dispatch mechanism:

- Element accepts visitor (first dispatch)
- Visitor visits element (second dispatch)
- Correct operation selected based on both types

Example: `ConcreteElement.accept(concreteVisitor)` calls `concreteVisitor.visitConcreteElement(this)`.

TL;DR: Two-phase dispatch: element visitor specific operation.

Keyword/Key mappings: Double dispatch | Type-specific operations | Accept/Visit methods | Element/Visitor interaction | Method resolution | Polymorphism

Q67: What are the different implementations of the Factory Pattern?

Answer: Factory Pattern has several variations: Simple Factory, Factory Method, and Abstract Factory.

Polished Answer:

Simple Factory:

- Single factory class with conditional logic
- Not a GoF pattern but common
- Example: `ShapeFactory.getShape("circle")`

Factory Method:

- Factory method in creator class
- Subclasses override to create products
- Example: `Document.createPage()`

Abstract Factory:

- Creates product families
- Multiple factory methods
- Example: GUIFactory.createButton() + createMenu()

TL;DR: Simple Factory = conditional; Factory Method = inheritance; Abstract Factory = families.

Keyword/Key mappings: Simple/Factory/Abstract | Conditional creation | Inheritance | Product families | GoF patterns | Variation

Q68: How does the Observer Pattern implement one-to-many dependency?

Answer: Observer Pattern establishes one subject with many observers, all notified when subject state changes.

Polished Answer: Implementation:

- Subject maintains observer list
- Subject state change triggers notify()
- Each observer receives update()
- Observers can attach/detach dynamically

Example: Stock price (subject) notifying multiple display panels (observers) on price change.

TL;DR: One subject, many observers. All notified on state change.

Keyword/Key mappings: One-to-many | Subject-Observer | Notification | Attach/Detach | Stock tracking | Dynamic dependencies

Q69: What is the difference between class-level and object-level structural patterns?

Answer: Class-level patterns use inheritance; object-level patterns use composition for structuring relationships.

Polished Answer:

Class-level:

- Use inheritance
- Fixed at compile time

- Example: Adapter using multiple inheritance

Object-level:

- Use composition
- Changeable at runtime
- More flexible
- Example: Decorator wrapping objects

Most structural patterns are object-level. Class-level patterns are rare.

TL;DR: Class = inheritance; Object = composition. Object-level more flexible.

Keyword/Key mappings: Inheritance vs Composition | Compile time vs Runtime | Flexibility
| Adapter vs Decorator | Structural patterns | Static vs Dynamic

Q70: How does the Command Pattern decouple invoker from receiver?

Answer: Command Pattern decouples invoker from receiver through command objects that encapsulate requests.

Polished Answer: Decoupling mechanism:

- Invoker only knows command interface
- Command encapsulates receiver reference
- Receiver unaware of invoker
- Command handles execution delegation

Example: Remote control (invoker) sends command objects; commands interact with devices (receivers).

TL;DR: Command object between invoker and receiver. Neither knows the other.

Keyword/Key mappings: Invoker/Receiver separation | Command interface | Encapsulation
| Remote control | Request handling | Decoupling

Q71: What is the Memento Pattern's role in preserving encapsulation?

Answer: Memento Pattern preserves encapsulation by storing object state without exposing internal details.

Polished Answer: Encapsulation preservation:

- Originator creates memento with state
- Memento accessible only to originator
- Caretaker stores memento without accessing internals
- State restored without breaking encapsulation

Example: Text editor saves state as memento; editor restores without external code knowing internal structure.

TL;DR: Save state without exposing internals. Originator controls state access.

Keyword/Key mappings: State hiding | Encapsulation | Originator control | State snapshot | Text editor | Internal details protection

Q72: How does the Abstract Factory Pattern support product families?

Answer: Abstract Factory ensures products from the same family are compatible and created together.

Polished Answer: Product family support:

- Factory methods create related products
- All products from same factory are compatible
- Switching factories switches entire family
- Prevents mixing incompatible products

Example: WindowsFactory creates WindowsButton + WindowsMenu; MacFactory creates Mac equivalents.

TL;DR: Related products created together. No incompatible mixing.

Keyword/Key mappings: Product families | Compatibility | Factory switching | UI components | Related products | Consistency

Q73: What is the difference between Observer Pattern and Publisher-Subscriber Pattern?

Answer: Observer and Pub-Sub are similar, but Pub-Sub adds an event bus/channel for more decoupling.

Polished Answer:

Observer:

- Subject directly references observers
- Synchronous notification
- Tighter coupling

Pub-Sub:

- Event bus between publishers and subscribers
- Asynchronous possible
- More decoupled
- No direct reference

TL;DR: Observer = direct; Pub-Sub = via event bus. Pub-Sub more decoupled.

Keyword/Key mappings: Direct vs Indirect | Event bus | Synchronous vs Asynchronous | Coupling | Message broker | Notification mechanism

Q74: How does the Flyweight Pattern handle intrinsic and extrinsic state?

Answer: Flyweight Pattern separates intrinsic state (shared) from extrinsic state (context-specific).

Polished Answer:

Intrinsic State:

- Shared across instances
- Immutable
- Stored in flyweight object
- Example: Character font, style

Extrinsic State:

- Varies per instance
- Passed to methods
- Stored by client
- Example: Character position, content

Separation enables sharing while maintaining flexibility.

TL;DR: Share immutable state; pass varying state. Maximizes memory efficiency.

Keyword/Key mappings: Intrinsic/Extrinsic | Shared vs Varying | Immutable state | Memory optimization | Text editor example | State separation

Q75: What is the difference between Data Transfer Object (DTO) and Value Object patterns?

Answer: DTO transfers data between layers; Value Object represents an immutable value.

Polished Answer:

DTO:

- Transfers data across boundaries
- Mutable, simple attributes
- No business logic
- Example: UserDTO with getters/setters

Value Object:

- Represents domain value
- Immutable
- Validated on creation
- Example: Money (amount + currency)

TL;DR: DTO = data transport; Value Object = immutable domain value.

Keyword/Key mappings: Data transfer | Immutability | Layer boundaries | Domain modeling | Getters/Setters | Validation

Q76: How does the Dependency Injection pattern implement DIP?

Answer: Dependency Injection implements DIP by injecting abstractions into classes, reversing dependency flow.

Polished Answer: DI implementation:

- Constructor injection: Dependencies passed via constructor
 - Setter injection: Dependencies set via setters
 - Interface injection: Interface provides setter
-

Example:

java

```
javascript.snippet JAVASCRIPT

class PaymentService {    private PaymentProcessor processor; // abstraction
    PaymentService
    (PaymentProcessor processor) { // injected        this.processor = processor;    }}
```

TL;DR: Inject abstractions, not create them. Reverses dependency direction.

Keyword/Key mappings: Constructor injection | Setter injection | Abstraction injection | Dependency reversal | DIP | Loose coupling

Q77: What is the Active Object Pattern?

Answer: Active Object Pattern decouples method execution from invocation using concurrent objects with synchronized message queues.

Polished Answer: Components:

- **Proxy:** Client-facing interface
- **Method Request:** Encapsulates method call
- **Activation Queue:** Buffers requests
- **Scheduler:** Executes requests
- **Servant:** Actual implementation

Use case: Concurrent systems where method calls from different threads need sequential execution.

TL;DR: Asynchronous method execution with queued requests. Decouples invocation from execution.

Keyword/Key mappings: Concurrency | Message queue | Async execution | Thread safety | Method requests | Decoupling

Q78: How does the Monitor Object Pattern ensure thread safety?

Answer: Monitor Object Pattern provides thread-safe access using synchronized methods within a monitor object.

Polished Answer: Implementation:

- Monitor lock acquired before method execution
- Only one thread inside monitor at a time
- Condition variables for thread cooperation
- Encapsulates synchronization logic

Example: Thread-safe queue with synchronized enqueue/dequeue methods.

TL;DR: Synchronized object access. One thread at a time in monitor.

Keyword/Key mappings: Thread safety | Synchronization | Monitor lock | Condition variables | Mutex | Concurrent access

Q79: What is the difference between Facade and Adapter patterns?

Answer: Facade creates a new simplified interface; Adapter makes existing interfaces compatible.

Polished Answer:

Facade:

- New unified interface
- Hides subsystem complexity
- Multiple subsystems behind one interface
- Example: Home theater facade

Adapter:

- Converts existing interfaces
- Makes incompatible interfaces work
- Single adaptee typically
- Example: Media adapter

TL;DR: Facade = simplify; Adapter = convert.

Keyword/Key mappings: Simplification vs Conversion | New vs Existing interface | Subsystem hiding | Compatibility | Home theater vs Media | Different intents

Q80: How does the Interpreter Pattern work?

Answer: The Interpreter Pattern defines a grammar and interprets sentences in that grammar using expression classes.

Polished Answer: Components:

- **Abstract Expression:** Common interface
- **Terminal Expression:** Grammar leaf nodes
- **Non-terminal Expression:** Composite expressions
- **Context:** Global information

Use cases: SQL parsing, regular expressions, mathematical expressions, DSL interpreters.

Example: Parsing "3 + 5" into NumberExpression(3), NumberExpression(5), AddExpression.

TL;DR: Parse and interpret grammar using expression tree. Useful for DSLs.

Keyword/Key mappings: Grammar parsing | Expression tree | DSL | SQL parser | Regular expressions | Syntax interpretation

Q81: What is the difference between Aggregation and Composition in design patterns?

Answer: Aggregation is a weak "has-a" relationship; Composition is a strong "owns-a" relationship.

Polished Answer:

Aggregation:

- Object can exist independently
- Weak ownership
- Example: Department has Students (students exist without department)

Composition:

- Object cannot exist without parent
- Strong ownership
- Lifecycle tied to parent
- Example: House has Rooms (rooms don't exist without house)

Design patterns often use composition for flexibility.

TL;DR: Aggregation = independent parts; Composition = dependent parts.

Keyword/Key mappings: Has-a vs Owns-a | Lifecycle | Independence | Strength of

Q82: How does the Strategy Pattern differ from the State Pattern in implementation?

Answer: Strategy Pattern has client-selected strategies; State Pattern has context-managed state transitions.

Polished Answer:

Strategy Implementation:

- Client sets strategy explicitly
- Context unaware of strategy changes
- No state transition logic
- Example: `context.setStrategy(new BubbleSort())`

State Implementation:

- Context changes state internally
- State transitions managed by states
- Each state knows next state
- Example: Vending machine transitions on coin insertion

TL;DR: Strategy = external selection; State = internal transitions.

Keyword/Key mappings: External vs Internal | Client vs Context | Strategy setting | State transitions | Algorithm vs Behavior | Explicit vs Automatic

Q83: What are the advantages and disadvantages of the Singleton Pattern?

Answer: Singleton provides single instance and global access but creates global state and testing issues.

Polished Answer:

Advantages:

- Single instance guaranteed
- Global access point
- Lazy initialization possible

- Resource conservation

Disadvantages:

- Global state (testing difficulty)
- Tight coupling
- Thread safety complexity
- Limits flexibility

Example: Database connection singleton convenient but hard to mock in tests.

TL;DR: Convenient global access but creates coupling and testing challenges.

Keyword/Key mappings: Single instance | Global access | Lazy init | Global state | Testing difficulty | Thread safety

Q84: How does the Prototype Pattern support dynamic object creation?

Answer: Prototype Pattern creates objects at runtime by cloning prototypes, enabling dynamic object types.

Polished Answer: Dynamic creation aspects:

- New object types at runtime
- No compile-time class requirement
- Object registration and lookup
- Flexible object hierarchies

Example: Game registering enemy prototypes at startup; cloning at runtime based on level requirements.

TL;DR: Clone prototypes at runtime. No need for compile-time class knowledge.

Keyword/Key mappings: Runtime creation | Cloning | Dynamic types | Registration | Game objects | Flexibility

Q85: What is the difference between Factory Pattern and Builder Pattern?

Answer: Factory creates objects in one step; Builder creates objects step-by-step.

Polished Answer:

Factory:

- Single method call creates object
- Focus on polymorphic creation
- Implementation determined by factory
- Example: `factory.createShape("circle")`

Builder:

- Multiple steps configure object
- Focus on complex construction
- Step-by-step field setting
- Example: `builder.name("x").age(30).build()`

TL;DR: Factory = one-step creation; Builder = multi-step construction.

Keyword/Key mappings: One-step vs Multi-step | Polymorphic vs Complex | Object configuration | Shape creation | Step-by-step | Different problems

Q86: How does the Chain of Responsibility differ from Decorator Pattern?

Answer: Chain of Responsibility passes requests through handlers; Decorator adds responsibilities to objects.

Polished Answer:**Chain of Responsibility:**

- Request passes through chain
- Handler may or may not process
- Stops when handled
- Example: Approval workflow

Decorator:

- Each decorator adds behavior
- All decorators process
- No stopping
- Example: Adding bold + italic + spell-check

TL;DR: Chain = one handler processes; Decorator = all add behavior.

Keyword/Key mappings: Request handling vs Behavior addition | Stop vs Continue |

Q87: What is the difference between Microservices and Monolithic architecture?

Answer: Microservices decompose application into small, independent services; Monolithic builds as single unit.

Polished Answer:

Microservices:

- Small, focused services
- Independent deployment
- Different technologies possible
- Complex distributed systems
- Example: Separate user, order, payment services

Monolithic:

- Single codebase
- Single deployment
- Simpler initially
- Scalability challenges
- Example: Traditional e-commerce app

TL;DR: Microservices = distributed small services; Monolithic = single unit.

Keyword/Key mappings: Service decomposition | Independent deployment | Single codebase | Scalability | Distributed systems | Architecture style

Q88: How does the Repository Pattern abstract data access?

Answer: Repository Pattern mediates between domain and data layers, abstracting data access logic.

Polished Answer: Benefits:

- Centralizes data access logic
- Decouples domain from data source
- Testable with mock repositories

- Consistent data access interface

Example: UserRepository with methods `findById()`, `save()`, `delete()` hiding database details.

TL;DR: Data access abstraction. Domain uses repository interface, not database directly.

Keyword/Key mappings: Data access | Abstraction | Domain/Data separation | Mocking | CRUD operations | Testability

Q89: What is the difference between Orchestration and Choreography in microservices?

Answer: Orchestration has a central coordinator; Choreography has services communicate directly.

Polished Answer:

Orchestration:

- Central coordinator manages workflow
- Clear flow control
- Single point of failure
- Example: Order service coordinates payment, inventory, shipping

Choreography:

- Services communicate via events
- No central control
- More resilient but complex
- Example: Services react to published events

TL;DR: Orchestration = central control; Choreography = decentralized events.

Keyword/Key mappings: Central vs Distributed | Coordinator | Event-driven | Control flow | Resilience | Microservice patterns

Q90: How does the Circuit Breaker Pattern improve system resilience?

Answer: Circuit Breaker Pattern prevents cascading failures by stopping calls to failing services.

Polished Answer: States:

- **Closed:** Normal operation, calls allowed
- **Open:** Failures exceed threshold, calls blocked
- **Half-Open:** Testing if service recovered

Benefits: Prevents resource exhaustion, graceful degradation, faster recovery.

Example: Payment service failures cause circuit breaker to open, preventing repeated calls.

TL;DR: Stop calling failing service. Prevents cascading failures.

Keyword/Key mappings: Failure prevention | Circuit states | Graceful degradation |
Microservices | Resilience | Cascading failures

Q91: What is the difference between Vertical and Horizontal Scaling?

Answer: Vertical scaling adds resources to single machine; Horizontal scaling adds more machines.

Polished Answer:

Vertical (Scale Up):

- More CPU, RAM to existing server
- Simpler implementation
- Hardware limits
- Example: Upgrading server specs

Horizontal (Scale Out):

- More servers
- Distributed systems
- Better for large scale
- Example: Adding more web servers

TL;DR: Vertical = bigger machine; Horizontal = more machines.

Keyword/Key mappings: Scale up vs Scale out | Resource addition | Server count |
Hardware limits | Distributed systems | Scalability

Q92: How does the Backpressure Pattern handle overload?

Answer: Backpressure Pattern manages overload by signaling producers to slow down when consumers can't keep up.

Polished Answer: Implementation:

- Consumer signals producer
- Queue depth monitoring
- Rate limiting
- Reactive streams

Example: Kafka consumers signaling producers to pause when processing lag is high.

TL;DR: Consumer controls production rate. Prevents buffer overflow.

Keyword/Key mappings: Load management | Rate control | Producer/Consumer | Reactive streams | Queue monitoring | Overload prevention

Q93: What is the difference between API Gateway and Service Mesh?

Answer: API Gateway handles external client requests; Service Mesh manages internal service-to-service communication.

Polished Answer:

API Gateway:

- Entry point for external clients
- Authentication, routing, rate limiting
- North-south traffic
- Example: Kong, Zuul

Service Mesh:

- Internal service communication
- Observability, retries, security
- East-west traffic
- Example: Istio, Linkerd

TL;DR: Gateway = external entry; Service Mesh = internal communication.

Keyword/Key mappings: External vs Internal | Client requests | Service communication | North-south vs East-west | Authentication | Observability

Q94: How does the Bulkhead Pattern isolate failures?

Answer: Bulkhead Pattern isolates failures by partitioning resources into separate pools.

Polished Answer: Implementation:

- Separate thread pools per service
- Resource isolation
- Prevents one failure affecting all
- Ship bulkhead analogy

Example: Separate connection pools for different microservices; one service's failure doesn't exhaust others' resources.

TL;DR: Isolate resources per service. One failure doesn't sink the ship.

Keyword/Key mappings: Resource isolation | Thread pools | Failure containment | Ship analogy | Connection pools | Microservices

Q95: What is the difference between CQRS and CRUD?

Answer: CRUD uses same model for read/write; CQRS separates read and write models.

Polished Answer:

CRUD:

- Single model for all operations
- Simple implementation
- Read/write consistency
- Example: Traditional database operations

CQRS:

- Separate read and write models
- Optimized for each
- Eventual consistency
- Complex queries possible
- Example: Read model denormalized for queries

TL;DR: CRUD = one model; CQRS = separate read/write models.

Keyword/Key mappings: Read/Write separation | Model optimization | Eventual consistency

Q96: How does the Event Sourcing Pattern work?

Answer: Event Sourcing stores state changes as a sequence of events rather than current state.

Polished Answer: Implementation:

- Every change captured as event
- Current state reconstructed from events
- Complete audit trail
- Temporal queries possible

Example: Bank account balance derived from transaction events, not stored directly.

TL;DR: Store events, reconstruct state. Complete audit history.

Keyword/Key mappings: Event log | State reconstruction | Audit trail | Temporal queries | Bank transactions | Historical data

Q97: What is the difference between Synchronous and Asynchronous communication?

Answer: Synchronous waits for response; Asynchronous continues without waiting.

Polished Answer:

Synchronous:

- Caller blocks until response
- Request-response model
- Simpler but couples services
- Example: REST API call

Asynchronous:

- Caller continues immediately
- Message queues, events
- Decoupled, resilient
- Example: Kafka event publishing

TL;DR: Sync = wait for response; Async = fire and forget.

Q98: How does the Idempotent Consumer Pattern handle duplicate messages?

Answer: Idempotent Consumer Pattern ensures duplicate message processing doesn't cause duplicate effects.

Polished Answer: Implementation:

- Track processed message IDs
- Deduplication logic
- Natural idempotency (e.g., setting value)
- Database unique constraints

Example: Payment service checking transaction ID before processing payment.

TL;DR: Safe to process same message multiple times. No duplicate effects.

Keyword/Key mappings: Duplicate handling | Message deduplication | At-least-once delivery | Transaction ID | Idempotency | Safe processing

Q99: What is the difference between Strong and Eventual Consistency?

Answer: Strong consistency guarantees immediate consistency; Eventual consistency allows temporary inconsistency.

Polished Answer:

Strong Consistency:

- All reads see latest write
- Immediate consistency
- Higher latency
- Example: Single database

Eventual Consistency:

- Reads may see stale data
 - Consistency achieved eventually
-

- Better availability
- Example: Distributed systems, DNS

TL;DR: Strong = immediate; Eventual = eventually consistent.

Keyword/Key mappings: Consistency models | Immediate vs Delayed | CAP theorem | Distributed systems | Latency | Availability

Q100: How does the Saga Pattern manage distributed transactions?

Answer: Saga Pattern manages distributed transactions as a sequence of local transactions with compensation.

Polished Answer: Implementation:

- **Choreography:** Services publish events
- **Orchestration:** Central coordinator manages saga
- **Compensation:** Undo actions if saga fails
- **Isolation:** Local transactions per service

Example: Order saga: create order reserve inventory process payment; compensation if payment fails.

TL;DR: Distributed transactions via local transactions + compensation. No distributed locks.

Keyword/Key mappings: Distributed transactions | Compensation | Choreography | Orchestration | Order management | Microservices

Summary Table: Pattern Categories & Interview Frequency

Category	Pattern	Frequency
Creational	Singleton	Very High
	Factory Method	Very High
	Builder	High
	Abstract Factory	High

	Prototype	Medium
Structural	Adapter	Very High
	Decorator	Very High
	Facade	High
	Proxy	High
	Bridge	Medium
	Composite	Medium
	Flyweight	Medium
Behavioral	Observer	Very High
	Strategy	Very High
	Command	High
	Template Method	High
	State	Medium
	Chain of Responsibility	Medium
	Visitor	Medium
	Iterator	Medium
	Mediator	Low
	Memento	Low
	Interpreter	Low

This document covers 100 design pattern interview questions organized by importance, with detailed answers, polished explanations, TL;DR summaries, and keyword mappings for efficient study.
